

2026 夏季学期

《软件工程基础实践》任务安排

这十天做什么、做到什么程度、老师怎么检查

10 天集中实践	前后两个阶段	项目 90 分	考勤 10 分
----------	--------	---------	---------

先看这一句 这十天不是另做一个新系统。你们要接着上学期的小组项目做：先把原系统整理好并跑通自动构建，再把后端拆成微服务，最后用测试证明它还能正常工作。

一、这次实践要完成什么

实践分成前后两个阶段，两个阶段都要做。90 分项目成绩按小组统一评分，作为全组共享的基础分，再根据每名成员的个人权重折算；10 分考勤按个人单独计算。

- 前 5 天（40 分）：确认上学期的系统还能启动，补齐缺少的文档和测试，把前端、后端、数据库装进容器，并跑通 CI/CD。
- 后 5 天（30 分）：把后端拆成至少 3 个业务微服务，自动部署到 Kubernetes，完成扩缩容、故障处理和性能对比。
- 项目管理、交付、演示和答辩（20 分）：最后一天现场演示、答辩并提交全部材料。

什么叫“补齐测试” 先看原项目已经有哪些测试、还缺哪些。缺的测试要补上，并在本机和流水线里实际跑通。不是多写几页测试文档，也不是只截一个“运行成功”的画面。

沿用上学期的项目

必须沿用上学期的小组和项目。第 1 天先把项目启动起来，列出完整的业务场景（用例）清单，并逐项检查是否能正常使用。

- 能正常启动：给当前版本打一个 Git 标签，建议命名为 `monolith-start`。这个版本不要再改，后面要用它和微服务版本做对比。
- 启动不了：先修到能运行，再继续后面的任务。确实无法修复的，当天向教师或助教说明。

为什么要保留这个版本 它就是你们改造前的原系统。验收时要证明：微服务版本完成了同样的主要业务，而且性能对比用的是同一套功能和数据。

二、所有业务场景（用例）都要完成，汇报时再选重点

第 1 天列出项目完整的业务场景（用例）清单，并由教师或助教确认。原项目中只有设想、尚未开发的内容，第 1 天说明后确认是否计入。确认后的清单就是最终验收范围。清单中的每个用例都要能运行，有对应的需求、设计、代码、测试和追溯记录。最终报告、答辩和现场演示可以从中选不少于 3 个有代表性的用例重点讲解，但其他用例也必须完成并接受检查。

什么算一个业务场景（用例） 由一类用户或外部系统触发，为完成一个明确的业务目标，从开始到产生可验证结果的一段完整流程。一个用例可以包含主成功流程、备选流程和异常流程，这些是同一用例的不同路径。单个页面、按钮、接口、数据库操作或流程中的一步，不单独算一个用例。

业务场景（用例）示例	不单独算一个业务场景（用例）
用户提交带库存和优惠规则的订单，并得到成功或失败结果；申请人提交申请，并得到审批结果。	只有登录、注册、查询一张表，或流程中的一个页面、按钮、接口、数据库操作。
用户完成一次包含多种状态和异常处理的预约，并得到预约结果。	如果登录或增删改查本身具有独立业务目标和复杂规则，可以单列用例，但要写清触发者、目标和结果。

清单中的每个业务场景（用例）都要有用例说明，至少写清参与者、触发条件、前置条件、主成功流程、备选或异常流程和可验证结果，并能一路对应到设计图、代码和测试。最终汇报时，至少选 3 个有代表性的用例详细展示这条对应关系。建议使用下面的编号方式：

对应关系示例 REQ01 / UC01 / SYS-SEQ01 / COMP-SEQ01 / OBJ-SEQ01 / UNIT-TC01 / INT-TC01 / E2E-TC01

三、前 5 天要做的事（40 分）

1. 把文档和图补齐

文档	至少要有
需求说明书	用户故事或需求编号、用例图、用例说明、概念类图。清单中的每个业务场景（用例）画 1 张系统级顺序图、状态图或活动图。
概要设计说明书	组件图。清单中的每个业务场景（用例）画 1 张组件级顺序图、状态图或活动图。
详细设计说明书	类图。清单中的每个业务场景（用例）画 1 张对象级顺序图、状态图或活动图。
追溯表	把需求、用例、三层模型、代码模块、测试编号和测试结果放在同一张表里。

2. 把测试补齐并跑通

测试	具体做什么	怎么算有效
单元测试	测试关键类、方法、业务规则和异常分支。	要有判断结果的断言，不能只看程序有没有报错。
集成/API 测试	测试模块之间的调用、数据库访问和对外接口。	各用例的主成功流程、备选流程和异常流程都要覆盖。
端到端测试	从页面或接口入口走完一个完整业务流程。	覆盖清单中的全部业务场景（用例）。

- 测试失败时，流水线必须停下来，不能继续发布镜像或部署。
- 测试报告要写清测试总数、通过数、失败数、失败原因和运行环境。
- 课程不统一规定代码覆盖率数字。覆盖率可以提交，但不能代替业务场景测试。

3. 把系统装进容器并跑通 CI/CD

- 前端、后端和数据库分别运行在容器中。数据库可以直接使用官方镜像，前端和后端要提交 Dockerfile。
- 准备数据库建表、数据迁移和测试数据脚本。换一台机器后，按 README 也能启动。
- 向指定分支 push 一次代码后，流水线自动完成：取代码、安装依赖、编译、单元测试、集成测试、制作镜像、部署到 Kubernetes、健康检查。
- 任何一步失败，后面的部署都不能继续。成功记录和失败记录都要保留。
- 提交流水线配置、Kubernetes 部署文件、测试脚本和部署脚本。镜像要有版本号，不能只使用 latest。

可以使用哪些工具 Jenkins、Gitea Actions、GitHub Actions、CodeArts、Gitee、云效等都可以。Kubernetes 可以使用 Docker Desktop、Minikube、Kind 或 k3d。

四、后 5 天要做的事（30 分）

1. 把后端拆成至少 3 个业务微服务

这里数的是业务服务。API 网关、注册中心、配置中心、前端和数据库不算在 3 个业务微服务里。不要按用例数量机械拆分，要说明每个服务负责什么业务、为什么这样分。

- 每个服务能单独构建、测试和部署。
- 每张业务表要明确归哪个服务管理。其他服务不能直接读写这张表。
- 可以在同一个数据库服务器中使用不同数据库或 Schema，但不能跨服务直接联表查询。
- 跨服务的数据操作通过接口、事件或消息完成，并说明调用失败时怎么处理。

举个例子 订单服务需要商品信息时，应调用商品服务接口，不能直接查询商品服务的数据库表。

需要提交：服务划分图、服务接口清单、数据表归属表、跨服务调用说明，以及改造前后的两个代码版本。

2. 让微服务也能自动构建和部署

- 代码提交后，相关微服务自动完成构建、测试、制作镜像和部署。
- 所有后端公开接口都要有 API 测试，清单中的全部业务场景（用例）都要做端到端回归测试。最终报告可重点展示不少于 3 个代表性用例的结果。
- 要能查看服务日志、健康检查、就绪检查和版本号，并现场说明一次部署失败是怎么查出来的。

3. 做两项云原生实验

实验	你们要做什么	现场要看到什么
自动扩缩容	设置资源请求/限制和 HPA，用压力测试让负载升高。	压力升高后 Pod 数量增加；压力下降后 Pod 数量减少。记录吞吐量、平均/P95 响应时间和错误率。

实验	你们要做什么	现场要看到什么
故障处理	主动停止一个依赖服务，或给它注入延迟。实现熔断、限流、超时返回、备用结果或故障隔离中的一种。	系统返回事先设计好的提示或备用结果，其他服务不跟着一起崩溃。

4. 对比改造前后的性能

选择 2 到 3 个主要接口。单体版本和微服务版本使用同一台机器、同一批数据、同一份压力脚本，并各运行至少 3 次。记录并发数、吞吐量、平均响应时间、P95 响应时间、错误率、CPU 和内存。

不要求微服务一定更快 如果微服务版本更慢，也可以。把测试条件和原始结果交出来，并解释为什么会有差异。只有确实测出来的结果，才能写“性能提升”。

五、每天怎么管理项目

每日站会

每天上午 9:00 到机房后，各组开 5 分钟左右的站会。每个人只回答三件事：昨天做了什么、今天做什么、现在卡在哪里。每天中午 12:00 前提交一份约 200 字的站会简报。云盘地址在课程群发布。

看板和任务

可使用 CodeArts、GitHub Projects、Gitee、云效、TAPD 或其他同类平台。教师和助教要能查看仓库和项目看板。每项任务至少写清：

字段	内容
任务内容	任务编号、名称、对应的需求或用例
谁来做	负责人，必要时写协作者
什么时候完成	计划时间、实际完成时间
做到什么算完成	验收条件
证据	对应的提交、Pull Request、测试或文档链接

- 每天保存一次看板和统计页面截图，并写上日期。最终报告只放有代表性的任务截图，其他任务由教师在线查看。
- 重要代码建议通过 Pull Request 合并，并由另一名组员检查。
- Git 提交次数不能直接当作个人贡献大小。还要看任务难度、质量、测试、文档和协作。

六、时间安排

时间	要完成的工作	检查
----	--------	----

时间	要完成的工作	检查
8月25日-8月28日	启动原系统；补文档和测试；完成容器化与CI；确定微服务怎么拆。	每天站会、看板和流水线记录
8月29日（周六）	正常上课，完成前5天任务。	中期检查
8月30日（周日）	不安排集中实践。	-
8月31日-9月3日	完成微服务、自动部署、回归测试、扩缩容、故障处理和性能对比。	每天站会、看板和实验记录
9月4日 14:00-17:00	最终答辩和现场演示。	最终验收

8月29日中期检查要看到什么

- 原系统能启动，确认后的业务场景（用例）清单中所有用例都能运行，并已打Git标签。
- 所有业务场景（用例）的需求、系统级图、组件级图、对象级图和追溯表已经完成。
- 前端、后端、数据库容器能够启动。CI能够自动完成构建和测试。
- 微服务划分图、服务接口清单和数据表归属方案已经完成。

未达到以上要求的组，在第6天上午提交修改结果。中期检查已经记录的完成情况仍会计入评分。

七、最终要交什么

1. 改造前的原系统版本和Git标签；改造后的微服务版本及完整提交记录。
2. README：写清环境版本、端口、启动方法、健康检查地址、测试账号和初始数据。
3. Dockerfile、流水线配置、Kubernetes YAML或Helm文件、数据库脚本和部署/回滚脚本。
4. 单元测试、集成/API测试、端到端测试代码，以及测试报告和流水线原始报告。
5. 压力测试脚本、原始结果、至少3次重复实验的数据和分析。
6. 完整的业务场景（用例）清单，以及需求、概要设计、详细设计、测试计划、测试报告和追溯表。
清单中标出最终汇报重点讲解的用例。提交可编辑文件、PDF和模型源文件。
7. 服务划分图、服务接口清单、数据表归属表和跨服务调用说明。
8. 项目管理平台地址、每日站会简报、每日看板/统计截图。
9. 答辩PPT、技术总结报告、个人权重表和全员确认记录。

目录	内容
01_source	代码或仓库清单
02_docs	需求、设计、测试、追溯和模型源文件
03_devops	Docker、流水线、Kubernetes/Helm和数据库脚本
04_tests	自动化测试、压力脚本、原始报告和实验数据
05_management	站会简报、看板截图、贡献与权重材料

目录	内容
06_defense	PPT、技术总结报告和备用演示材料

压缩包建议命名为“班级-组号-项目名-软件工程基础实践.zip”。最终截止时间 T 和云盘地址由课程群发布，原则上不晚于第 7 天通知；答辩顺序和模板地址也由课程群发布。

- 每迟交不足 24 小时按 24 小时计，从小组共享的 90 分项目成绩中扣 5 分，全组成员共同承担，最多扣 15 分。超过 72 小时原则上不再接收。
- 因权限、缺文件或说明不完整导致教师打不开或启动不了，按对应项目扣分。通知后补交，只恢复检查资格，不自动恢复已扣分。
- 网络或云平台故障时可以改为本地演示。建议准备 5 到 8 分钟备用录屏，但录屏不能代替代码、配置、日志和现场回答。

八、答辩怎么进行

每组原则上 15 分钟：项目和架构说明 4 分钟，现场演示 7 分钟，问答 4 分钟。每名成员都要到场，并可能被问到本人负责的工作。

- PPT 讲清任务完成情况、CI/CD 过程、微服务怎么拆、数据归谁管理、测试结果、扩缩容、故障处理和性能对比。
- 现场优先演示可重复的过程：push 一次代码触发流水线、查看 Kubernetes 部署、运行选定的重点讲解用例、触发扩缩容和停止一个依赖服务。其他业务场景（用例）仍可能被抽查。
- 不能只播放录屏或只放截图。使用备用材料时，仍要接受代码、配置和日志检查。

九、小组共享评分标准（项目 90 分）

除考勤外，下面所有评分项都以小组为单位验收并评分，形成一份全组共享的项目成绩。不会把 40 分、30 分和 20 分分别拆给个人，也不会按成员分工把某一评分项只算给某个人。表中的“综合评分”指项目管理、交付、演示和答辩。评分主要看三件事：系统是否真的能运行，过程能不能重复，材料能不能对应到代码和测试。缺少代码、配置、日志或原始报告，只放截图，不能按完整完成计分。

小组评分项	所属阶段	分值	小组拿满分需要做到
文档	第一阶段	10	所有业务场景（用例）的需求、三层模型、代码和测试能一一对应；最终汇报重点讲清至少 3 个代表性用例。
测试	第一阶段	10	三类测试都已运行，断言有效，报告完整，失败会让流水线停止。
容器化	第一阶段	8	前端、后端、数据库能按 README 启动，数据库能自动初始化。
原系统 CI/CD	第一阶段	10	push 后自动完成构建、测试、镜像、Kubernetes 部署和健康检查。
中期检查	第一阶段	2	按时达到中期检查要求。
微服务划分	第二阶段	7	至少 3 个业务微服务，划分理由和接口说明清楚。
数据归属	第二阶段	5	表归属明确，不跨服务直接访问表，调用失败有处理办法。
微服务流水线	第二阶段	5	相关服务能自动构建、测试、制作镜像和部署。

小组评分项	所属阶段	分值	小组拿满分需要做到
用例回归	第二阶段	4	所有公开接口和清单中的全部业务场景（用例）通过自动测试。
自动扩缩容	第二阶段	3	压力能触发扩容，负载下降后能缩容，记录完整。
故障处理	第二阶段	2	停止依赖服务后返回设计好的结果，其他服务没有一起崩溃。
性能对比	第二阶段	4	条件相同、至少运行 3 次，原始数据和解释完整。
项目管理	综合评分	4	站会、看板和任务证据持续更新。
交付和报告	综合评分	6	材料齐全、链接可访问、按 README 可以复现。
现场演示	综合评分	5	主要过程能现场运行，并能说明失败怎么处理。
答辩和问答	综合评分	5	成员能说明本人工作，也了解系统整体。

前 5 天任务共 40 分，后 5 天任务共 30 分，项目管理、交付、演示和答辩共 20 分，合计 90 分。这 90 分先形成全组共享的项目基础分，再用于计算每名成员的个人项目成绩。

不能运行时怎么计分

- 检查时原系统或微服务版本完全不能运行，对应的实现、测试和演示项目原则上最多得该项一半分数。
- 伪造截图、日志、提交记录、测试结果或他人签名，按学术诚信问题处理。
- 直接复制往届或其他小组项目，且不能说明来源和本人工作，相应项目可记 0 分。

十、个人成绩和考勤

先共享项目成绩，再按个人权重折算

计算方法 个人项目成绩 = $\min(90, \text{小组共享项目成绩} \times \text{个人权重})$ ；课程总成绩 = $\min(100, \text{个人项目成绩} + \text{个人考勤成绩})$ 。个人权重只作用于项目成绩；考勤按个人实际出勤单独计算，不共享、不加权。

- 一般情况下，个人权重在 0.9 到 1.1 之间。经教师和助教核实，特殊情况可低至 0.5。组长不会因为是组长自动加权。
- 全组权重之和不能超过组员人数。
- 权重方案原则上由全员签字或线上确认。意见不一致时，先按 1.0 计算，再由教师和助教根据看板、提交、评审、站会和答辩情况决定。

考勤（10 分）

- 每天上午、下午各考勤 1 次，共 20 次，考勤时间不固定。
- 普通日期未请假缺勤，前 2 次不扣分，从第 3 次起每次扣 1 分。第 1 天上午和第 10 天下午是关键考勤，每缺勤 1 次直接扣 1 分。
- 迟到或早退不超过 30 分钟记 1 次，累计 2 次折合 1 次缺勤；超过 30 分钟按 1 次缺勤。临时离开机房要报备。

- 病假、事假按学校和课程要求办理请假。是否安排补验由教师决定。

十一、AI 工具和学术诚信

- 可以使用生成式 AI 讨论方案、辅助写代码、生成测试、排查错误和修改文字。组员必须检查并实际运行，不能把 AI 输出直接当成正确答案。
- 技术总结报告中简要写明用了什么工具、用来做什么、怎样人工检查。
- 引用开源代码、模板、数据和教程时，保留许可证和来源。不得提交自己解释不了的代码。
- 仓库中不能出现密码、Token、数据库口令和云平台密钥。使用环境变量、Secret 或平台凭据管理。

十二、参考资料

[Docker 官方文档](https://docs.docker.com/) <https://docs.docker.com/>

[Kubernetes 官方文档](https://kubernetes.io/docs/home/) <https://kubernetes.io/docs/home/>

[Kubernetes HPA 说明](https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/) <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>

[GitHub Actions 文档](https://docs.github.com/en/actions) <https://docs.github.com/en/actions>

[Jenkins 用户文档](https://www.jenkins.io/doc/) <https://www.jenkins.io/doc/>